

Web Application Security

Study Summary – Chapters 1–13

Applied College | Web Security Course

Textbook: *Web Application Security* — Andrew Hoffman, O'Reilly, 2020

Ch1: History of Software Security • Ch2: Web App Reconnaissance • Ch3: Finding Subdomains
Ch4: API Analysis • Ch5: Third-Party Dependencies • Ch6: Weak Points in Architecture
Ch7: Hacking Web Applications • Ch8: Web Security • Ch9: Secure Architecture
Ch10: Reviewing Code • Ch11: Vulnerability Discovery • Ch12: Vulnerability Management
Ch13: Defending Against XSS, CSRF & XXE

Prepared: June 2026

Chapter 1: The History of Software Security

Key Definitions

Software Security: A set of practices that help **protect** software applications and digital solutions from **attackers**, incorporated into the software development life cycle and testing processes.

Hacking: An attempt to **exploit** a computer system or private network — the **unauthorized** access to or control over computer network **security** systems for **illicit** purposes.

Enigma Machine (circa 1930): An electric-powered mechanical rotor device used to **encrypt** and **decrypt** text messages. It used a symmetric **key** algorithm — **encrypting** and **decrypting** with a single **cryptographic key**.

Symmetric Key Algorithm: A cipher that uses a single **cryptographic key** for both **encryption** and **decryption**. Used by the Enigma machine; still used today for **securing** data in transit.

Turing Test: A test developed by Alan Turing to rate conversations generated by machines based on how difficult it is to differentiate them from human conversations. A foundational concept in AI.

Morris Worm (1988): One of the **first** recognized internet worms; caused widespread **damage** and led to the creation of CERT (Computer Emergency Response Team) — the **first** formal **security** incident response team.

CERT (Computer Emergency Response Team): The **first** formal **security** incident response organization, established in response to the Morris Worm. Marked the beginning of organized software **security** as a discipline.

Exam Tip: The Enigma machine is the origin of modern symmetric-key **cryptography**. Symmetric = one **key** for both **encryption** and **decryption**.

Exam Tip: Alan Turing is associated with two major contributions: (1) breaking the Enigma code (WWII), (2) the Turing Test (AI foundation).

Exam Trap: Symmetric **key** ≠ asymmetric **key**. Symmetric uses ONE **key**; asymmetric (public **key cryptography**) uses TWO **keys** (public + private).

Exam Trap: CERT was created AFTER the Morris Worm (1988), **not before**. Formal **security** organizations did **not** exist **prior** to the worm.

Chapter 2: Introduction to Web Application Reconnaissance

Key Definitions

Web Application Reconnaissance (Recon): The explorative data-gathering phase that generally occurs **prior** to **hacking** a web application. Performed by **hackers**, pen testers, or **security** engineers to find **weakly secured** mechanisms.

Information Gathering: Investigating the target using publicly available information. Includes: organization website, WHOIS databases, ARIN (IP ownership), and DNS records.

WHOIS Database: A public registry containing information about domain owners including names, telephone numbers, and email addresses — a primary reconnaissance source.

ARIN (American Registry for Internet Numbers): A database that provides information about who owns a specific IP address or range of addresses. Used during recon to identify server owners.

Social Engineering: Deceiving people into revealing sensitive or useful information via phone, email, or impersonation. Results in **leaking** passwords, credit card numbers, and bank account details.

Dumpster Diving: An **attack** technique where an **attacker** searches through physical trash to find old documents, discarded disks, or post-it notes containing usernames, passwords, or other sensitive data.

Web Application Mapping: Building a map representing the structure, organization, and functionality of a web application. This **must** be the **first** step **before** attempting to **hack** a web application.

Exam Tip: Reconnaissance is **NOT only** performed by **attackers** — **security** engineers use the same recon techniques to find and **fix** weaknesses **before attackers** do.

Exam Tip: WHOIS + ARIN + DNS together provide a comprehensive picture of a target's network infrastructure.

Exam Trap: Web application mapping **must always** be the **FIRST** step — **never** attempt **hacking** without **first** mapping the target application.

Exam Trap: Social engineering **attacks** do **not** require **any** technical skill — they **exploit** human psychology, **not** software **vulnerabilities**.

Chapter 3: Finding Subdomains

Key Definitions

Subdomain: A subdivision of a primary domain used to host separate services (email, admin, file servers). Large organizations typically have many subdomains attached to their primary domain.

Zone Transfer Attack: A DNS **attack** technique that attempts to retrieve the entire DNS zone file from a misconfigured DNS server, **exposing all** subdomains and internal host records at once.

Brute Force / Dictionary Attack (Subdomains): Systematically guessing subdomain names using wordlists to discover undocumented or hidden subdomains that are **not** listed in public records.

Subdomain Recon Methods

- Browser's built-in Network Analysis Tools (DevTools → Network tab).
- Public Records (GitHub repos, **certificates**, WHOIS).
- Search Engine Caches (Google, Bing cached pages).
- Accidental Archives (**leaked** repos, **exposed** configuration files).
- Social Snapshots (Twitter API, LinkedIn).
- Zone Transfer **Attacks** (DNS misconfiguration **exploit**).
- Brute Forcing / Dictionary **Attacks** on subdomain names.

Exam Tip: The browser Network tab shows **all** HTTP requests including those to subdomains — a quick **first** step for mapping affiliated servers.

Exam Trap: Zone transfer is **only** possible on misconfigured DNS servers. Most modern servers **restrict** zone transfers — do **not** assume it will **always** work.

Chapter 4: API Analysis

Key Definitions

API Analysis: The second step in web application recon, following subdomain discovery. Provides information needed to understand the purpose and structure of an **exposed** API.

REST API: Representational State Transfer API; specifies resources rather than functions in endpoints, is stateless (server does **not** track requesters), and is hierarchical.

SOAP API: Simple Object Access Protocol; an older, more rigid API format. REST is now more popular and is the preferred structure for modern web applications.

OPTIONS Method: A special HTTP method that **only** exists to provide information about which HTTP verbs an API endpoint supports. The **first** go-to method when performing recon against an API.

Stateless API: A REST API design principle where the server does **not** keep track of its requesters. Each request **must** contain **all** the information needed to process it independently.

Authentication Schemes

Scheme	How it works
Session Cookie	Server creates a session ID stored in a browser cookie
JWT (JSON Web Token)	Signed token containing claims; sent with every request
OAuth 2.0	Token-based delegated authorization; used for third-party login
Basic Auth	Base64-encoded username:password sent in the Authorization header
API Key	Static key passed in header or query string to identify the caller

Exam Tip: Send an OPTIONS request **first** when probing an API — it reveals supported HTTP verbs without modifying **any** data.

Exam Trap: OPTIONS is **not always** available — most enterprise APIs disable it. Do **not** rely solely on OPTIONS for endpoint discovery.

Exam Trap: REST APIs are stateless — the server does **NOT** store session state. Do **not** confuse REST statelessness with SOAP's stateful sessions.

Chapter 5: Identifying Third-Party Dependencies

Key Definitions

Third-Party Dependency: External code integrated into a web application (proprietary or open-source). Often **not** subject to as robust a **security** review as in-house code; can become **attack** vectors.

CVE (Common Vulnerabilities and Exposures): A public database of known **security vulnerabilities**. If a **vulnerable** third-party dependency is **detected**, an **attacker** may copy an existing **attack** directly from the CVE database.

SPA Framework (Single Page Application): A client-side JavaScript framework that renders pages in the browser without full page reloads. Major SPA frameworks: React (Facebook), AngularJS (Google), VueJS (Adobe/GitLab), EmberJS.

Prototype Pollution: A JavaScript **vulnerability** where an **attacker** can modify the prototype of a base object, potentially affecting **all** instances and enabling unexpected code execution.

ReDoS (Regular Expression Denial of Service): An **attack** that **exploits** poorly written regular expressions that take exponential time to evaluate certain inputs, causing a **denial** of service.

Detecting SPA Frameworks

Framework	Detection Method
EmberJS	Global Ember object; DOM elements with <code>id=ember1</code> , <code>class="ember-application"</code>
AngularJS	Global <code>angular</code> or <code>ng</code> object; first root element has <code>ng-version</code> attribute
React	Global React object; <code><script type="text/jsx"></code> tags
VueJS	Global Vue object; <code>Vue.version</code> constant

Exam Tip: **Detecting** the exact version of a third-party library lets you search CVE databases for known **exploits** — even **before** writing custom **attacks**.

Exam Trap: Third-party dependencies are **NOT** automatically **safe** just because they are popular or open-source. **Always verify** versions against CVE databases.

Chapter 6: Identifying Weak Points in Application Architecture

Key Definitions

Architectural Vulnerability: Most **vulnerabilities** stem from improperly designed application architecture rather than from poorly written individual methods. Architecture **flaws** affect entire features, **not** just one function.

Multiple Vulnerabilities Indicator: A single **vulnerability** may result from bad code. Multiple **vulnerabilities** in the same application indicate weaknesses in the architecture itself.

Security Levels: **Secure:** implements **security prior** to and during feature development. Middle-level: partial **security**. **Insecure:** no **security** implementation at **all**.

XSS Risk via innerHTML: **Any** method that provides HTML directly to the DOM is a **risk** because it allows HTML to be uploaded (if **sanitization** is **not** provided) and can execute scripts on another user's machine.

Multiple Layers of Security: Layers for XSS **risk** in an IM application: (1) API POST, (2) Database Write, (3) Database Read, (4) API GET, (5) Client Read. Each layer **must** be **secured**.

Exam Tip: Look for multiple **vulnerabilities** of the same type — they signal a broken **security** architecture, **not** just a coding mistake.

Exam Trap: **Fixing** one **vulnerability** does **not fix** the architecture. If the architecture is **insecure**, new **vulnerabilities** will keep appearing in the same areas.

Chapter 7: Introduction to Hacking Web Applications

Key Definitions

XSS — Cross-Site Scripting: An **attacker** uploads **malicious** script code onto a website that can **steal** data or perform **damage**. Three types: Stored (in database), Reflected (**not** stored, reflected by server), DOM-based (in browser).

CSRF — Cross-Site Request Forgery: An **attack** that tricks an **authenticated** user into unknowingly submitting a **malicious** request. The server **cannot** distinguish between a legitimate request and a **forged** one.

XXE — XML External Entity: An **attack** against applications that parse XML input. Allows an **attacker** to view files on the server's file system, perform SSRF, or execute code by **injecting** external entity references.

SQL Injection: **Injecting malicious** SQL commands into input fields that are passed unsanitized into database queries, enabling **unauthorized** data access, modification, or deletion.

DoS — Denial of Service: An **attack** that makes a system or service unavailable by overwhelming it with requests or **exploiting vulnerabilities** (e.g., ReDoS — crafted regex input causing exponential processing time).

Hacker's Mindset: A **hacker** analyzes existing code to seek entry points rather than making them. A great **hacker** is organized, technically skilled, and constantly learning new techniques.

Attack	Designated by OWASP	Target
Stored XSS	Yes	Database → Client browser
Reflected XSS	Yes	Server response → Client browser
DOM-based XSS	Yes	Client-side JavaScript execution
CSRF	Yes	Authenticated user session
SQL Injection	Yes	Database layer
XXE	Yes	XML parser / Server file system

Exam Tip: OWASP designates Stored, Reflected, and DOM-based XSS as the three most common XSS **attack** vectors — know **all** three types.

Exam Trap: CSRF does **not steal** data — it forces the victim to perform an action. XSS **steals** data/executes code. Do **not** confuse their mechanisms.

Chapter 8: Web Security — Securing Modern Web Applications

Key Definitions

Defensive Software Architecture: **Security** begins **prior** to **any** code being written, in the architecture phase. It is much easier to catch and resolve deep architectural **security flaws before** writing code.

Comprehensive Code Reviews: **Every** commit should be reviewed for **security** standards **before** release. Code reviews **must** be performed **not only** by the committer's team but also by an unrelated team.

Bug Bounty Program: A program that incentivizes external **security** researchers to find and responsibly **disclose vulnerabilities** in exchange for monetary rewards, **before malicious** actors find them.

Red Team / Blue Team: Red team: internal **attackers** who try to **exploit** the application. Blue team: defenders who **detect** and respond to **attacks**. Together they improve **security** posture.

Regression Testing: Testing that ensures a previously **fixed vulnerability** has **not** been reintroduced in newer code. **Critical** because **fixes** can sometimes break other parts of the application or be undone.

Vulnerability Triage: The process of prioritizing discovered **vulnerabilities** based on **risk** level. **Not all vulnerabilities** carry equal **risk** — some can wait, others require immediate **patching**.

Exam Tip: The order in the book (and course) is: Recon → Offense → **Defense**. Understanding **attack** techniques is **essential** for building effective defenses.

Exam Tip: Code review should check: how data is transmitted, how data is stored, how data is presented to users, and what operations occur on the server.

Exam Trap: Code review alone is **NOT** enough — architecture review **must** come **first**, then code review, then **vulnerability** discovery. **Never** skip the architecture phase.

Exam Trap: Old-fashioned **vulnerability** discovery via customer notification or public **disclosure** is reactive — **always** prefer proactive methods (bug bounty, red team).

Chapter 9: Secure Application Architecture

Key Definitions

Architecture Phase — Cost of Security: NIST study finding: 'The cost of removing an application **security vulnerability** during the design phase is 30–60 times less than if removed during production.'

Data in Transit: **All** data sent over the network **must** be **encrypted** en route to **prevent** man-in-the-middle attacks. **Required:** SSL or TLS **before any** data is transmitted.

TLS vs SSL: TLS (Transport Layer **Security**) offers the most rigid **security**. SSL (**Secure** Sockets Layer) has higher adoption but multiple known **vulnerabilities**. HTTPS requires TLS/SSL **before** allowing **any** data to be sent.

Password Entropy: The amount of randomness and uncertainty in a password. **Secure** passwords have high entropy — pattern-based passwords are **weak** regardless of length.

Dictionary Attack: A brute-force technique where the **attacker** uses a list of the most common passwords to guess credentials. Most effective against low-entropy (predictable) passwords.

2FA — Two-Factor Authentication: Requires users to provide two forms of identification: something they know (password) and something they have (phone OTP) or something they are (biometric).

PII (Personally Identifiable Information): **Any** data that could be used to identify a specific individual. **Must** be stored with special **security** measures and handled according to legal regulations.

Credential Hashing: Passwords **must never** be stored in plaintext — they **must** be **hashed** using a strong one-way function (e.g., bcrypt, Argon2) with a unique **salt** per user.

Exam Tip: NIST recommends **fixing security** in the architecture phase — 30–60x cheaper than **fixing** in production. This is a **key** exam statistic.

Exam Tip: TLS is preferred over SSL. When the exam asks about **securing** data in transit, the answer is TLS (or HTTPS).

Exam Trap: Password length and special characters alone do **NOT** make a password **secure** — entropy (randomness/unpredictability) is what matters.

Exam Trap: **Never** store passwords in plaintext or with reversible **encryption**. **Always** use one-way **hashing** with **salt**.

Chapter 10: Reviewing Code for Security

Key Definitions

Security Code Review: Checks for common **vulnerabilities** (XSS, CSRF, **injection**) AND logic-level **vulnerabilities** that require deep understanding of the code's purpose. **Always** occurs after the architecture review.

Logic Bug: A **vulnerability** that arises from flawed business logic rather than a common coding mistake. **Cannot** be found by automated tools — requires understanding the feature's purpose.

Blacklist (Anti-Pattern): A temporary **security** measure that **only blocks** known bad inputs. **Only protects** if you have perfect knowledge of **ALL** current and future **malicious** inputs — impossible in practice.

Whitelist (Preferred): Explicitly defines what IS allowed, rejecting everything else. More **secure** than a **blacklist** because it does **not** rely on enumerating **all** possible **threats**.

Code Review Flow: 1. git checkout master, 2. git pull origin master, 3. git checkout feature-branch, 4. git diff origin/master — shows changed files and diffs against master.

Where to Start a Security Review: 1. Client-side code (understand business logic), 2. API layer, 3. Dependencies, 4. Unintentionally **exposed** public APIs, 5. Remainder of codebase.

Exam Tip: Code review **must always** occur AFTER the architecture review — **never before**. This is the correct order.

Exam Tip: A **whitelist** is **always** preferred over a **blacklist** for input **validation** — it is impossible to know **every** possible future **malicious** input.

Exam Trap: Blacklists give a **false** sense of **security** — they can almost **always** be **bypassed** by an **attacker** who knows the pattern. **Never** rely on them as a primary **defense**.

Exam Trap: Automated scanners **cannot** find logic bugs — they **only** find archetypical **vulnerabilities**. Human review is **always required** for logic-level **security**.

Chapter 11: Vulnerability Discovery

Key Definitions

Security Automation: The **first** step after architecture and code reviews in discovering **vulnerabilities**. Automation is cheap, effective, and long-lasting. Three forms: static analysis, dynamic analysis, regression testing.

Static Analysis: Scripts that analyze source code for syntax errors and common **vulnerabilities** without executing code. Can be run locally, on-demand, or on each commit. **NOT** suitable for dynamically typed languages.

Dynamic Analysis: Tests the running application for **vulnerabilities**. Executes code and observes behavior. Can **detect** issues that **only** appear at runtime (e.g., race conditions).

Vulnerability Regression Testing: Automated tests that **verify** previously **fixed vulnerabilities** have **not** been reintroduced. **Essential** for ensuring that **fixes** stay **fixed** across new releases.

Static Analysis Tools

Tool	Language	Cost
Checkmarx	Most major languages	Paid
PMD	Java	Free
Bandit	Python	Free
Brakeman	Ruby	Free

What Static Analysis Can Detect

- General XSS: Look for DOM manipulation with innerHTML.
- Reflected XSS: Look for variables pulled from URL parameters.
- DOM XSS: Look for **dangerous** DOM sinks like setInterval().
- SQL **Injection**: Look for user-provided strings in database queries.
- CSRF: Look for state-changing GET requests.
- **DoS**: Look for improperly written regular expressions.

Exam Tip: Static analysis is best for statically typed languages (Java, C#). It struggles with dynamically typed languages like JavaScript.

Exam Trap: Automation **cannot detect** logical **flaws** or **vulnerability** chaining (multiple **weak vulnerabilities** combined into one strong **exploit**). **Always** supplement with manual review.

Chapter 12: Vulnerability Management

Key Definitions

SSDL (Secure Software Development Life Cycle): A well-defined pipeline for obtaining, triaging, and resolving **vulnerabilities** found in a web application, from the architecture phase through production.

Staging Environment: An environment that mimics the production environment as closely as possible, used to reproduce and **verify vulnerabilities** without affecting real users.

Reproducing a Vulnerability: The **first** step in managing a **vulnerability** report. Allows you to: **verify** it is a real **vulnerability**, avoid paying bug bounty for **false** positives, and gain insight for **fixing** it.

CVSS (Common Vulnerability Scoring System): A freely published system for ranking **vulnerabilities** based on how easy they are to **exploit** and what type of data or processes can be **compromised**. Composed of Base, Temporal, and Environmental metrics.

CVSS Base Metrics: Represent the intrinsic characteristics of a **vulnerability** that are **constant** over time and across environments. Composed of Exploitability metrics and Impact metrics.

CVSS Temporal Metrics: Score the **severity** of a **vulnerability** over time — accounts for **availability** of **exploits, patches**, and workarounds that may change the effective **risk**.

CVSS Environmental Metrics: Score a **vulnerability** based on the specific environment it exists in — considers the organization's **security** controls and the criticality of affected systems.

Risk Factors for Vulnerability Prioritization: Financial **risk** to the company, difficulty of exploitation, type of data **compromised**, existing contractual agreements, and **mitigation** measures already in place.

Exam Tip: CVSS has THREE metric groups: Base (**constant**), Temporal (changes over time), Environmental (context-specific). Know **all** three.

Exam Tip: **Always** reproduce a **vulnerability** in a staging environment **before** triaging — a staging environment mimics production but has no real user data.

Exam Trap: **Not all vulnerabilities** need immediate **fixes** — **risk** assessment determines priority. **Never** treat **all vulnerabilities** with equal urgency.

Exam Trap: Lack of logging for known **vulnerabilities** has caused company failures — **always** add logging for known **vulnerabilities** while the **fix** is being developed.

Chapter 13: Defending Against XSS, CSRF & XXE

Key Definitions

XSS Root Cause: Cross-site scripting is mainly caused by the failure of a web application to **sanitize** user inputs embedded in web pages. Conditions: app accepts user input, input creates dynamic content, input is insufficiently **validated**.

Anti-XSS Golden Rule: 'Do **not** allow **any** user-supplied data to be passed into the DOM — except as strings.' Use `innerText` (**not** `innerHTML`) whenever appending string-like objects to the DOM.

innerText vs innerHTML: `innerText`: interprets content as plain text — **safe** for user-supplied data. `innerHTML`: interprets content as HTML/JS — **dangerous** with user-supplied data. **Always** prefer `innerText` unless HTML rendering is explicitly **required**.

Input Sanitization: Removing or escaping **dangerous** content from user input **before** processing or displaying it. **Must** allow some HTML tags (e.g., `<strong>`) but **block dangerous** ones (e.g., `<script>`).

DOMParser: A JavaScript interface that parses XML or HTML source code from a string into a DOM Document.

Risk: loading untrusted content via `parseFromString` can execute **malicious** scripts.

CSRF Token: A unique, secret, unpredictable **token** generated per session and embedded in forms. The server **validates** the **token** on **every** state-changing request — an **attacker cannot forge** it.

SameSite Cookie Attribute: A cookie attribute that **prevents** the browser from sending the cookie in cross-site requests. `SameSite=Strict` or `SameSite=Lax` significantly reduces CSRF **attack** surface.

XXE Defense: Disable external entity processing in the XML parser. **Never** allow user-supplied XML to define or reference external entities.

SQL Injection Defense: Use parameterized queries (prepared statements) exclusively. **Never** concatenate user-supplied strings directly into SQL queries.

Content Security Policy (CSP): An HTTP response header that tells browsers which sources of content are legitimate. **Prevents** XSS by **blocking** inline scripts and scripts from untrusted origins.

Exam Tip: `innerText` is **always** the **safe** choice for displaying user content in the DOM. `innerHTML` should **only** be used when HTML rendering is absolutely necessary.

Exam Tip: CSRF **tokens** + SameSite cookies together provide strong CSRF **protection**. Use both.

Exam Trap: **Sanitizing** HTML by blacklisting specific tags (like `<script>`) is insufficient — **attackers bypass** blacklists with encoding tricks. Use a **whitelist** or a trusted **sanitization** library.

Exam Trap: CSP does **NOT prevent all** XSS — it reduces the impact. Proper input **sanitization** is still **required**. **Never** rely on CSP alone.

Exam Trap: SQL **injection cannot** be **fixed** by input **filtering** alone — parameterized queries are the **only** complete **defense**. **Never** concatenate user input into SQL.